

```

1 # Part 2 Step by Step
2 # 1. Loading spatial datasets: taxi trips (points → lines), neighborhoods (polygons)
3 # 2. Converting everything to same coordinate system (EPSG 2263)
4 # 3. Turning trips into lines
5 # 4. Finding which neighborhoods each trip crosses
6 # 5. Computing: trip speed and average speed per neighborhood
7 # 6. Visualizing results on a choropleth map
8
9 import json
10 from pathlib import Path
11 import duckdb
12 import folium
13
14 BASE_DIR = Path(__file__).resolve().parent
15 DATA_DIR = BASE_DIR / "data"
16 DB_PATH = BASE_DIR / "taxi_analysis.duckdb"
17
18 # Constants
19 SHAPEFILE_PATH = DATA_DIR / "ZillowNeighborhoods-NY.shp"
20 FEET_PER_MILE = 5280.0
21 MAX_SPEED_MPH = 100
22 SECONDS_PER_HOUR = 3600.0
23
24 def load_spatial(conn: duckdb.DuckDBPyConnection) -> None:
25     conn.execute("INSTALL spatial;")
26     conn.execute("LOAD spatial;")
27
28 def main() -> None:
29     print("Part 2: Average taxi speed by crossed neighborhood")
30
31     conn = duckdb.connect(str(DB_PATH))
32     load_spatial(conn)
33
34     conn.execute(
35         f"""
36         CREATE OR REPLACE TABLE taxi_trips AS
37         SELECT *
38         FROM read_csv_auto('{DATA_DIR} / "nyc_taxi_data.csv"', header=true)
39         """
40     )
41
42     # Create neighborhoods table from shapefile, eliminates duplicated neighborhoods
43     conn.execute(
44         f"""
45         CREATE OR REPLACE TABLE neighborhoods_2263 AS
46         WITH raw_neighborhoods AS (
47             SELECT
48                 Name,
49                 City,
50                 ST_Transform(geom, 'EPSG:4326', 'EPSG:2263', always_xy := true) AS geom,
51                 ROW_NUMBER() OVER (PARTITION BY Name ORDER BY Name) AS name_rank
52             FROM ST_Read('{SHAPEFILE_PATH}')
53             WHERE City = 'New York'
54         )
55         SELECT
56             Name AS neighborhood,
57             geom
58         FROM raw_neighborhoods
59         WHERE name_rank = 1
60         """
61     )
62
63     # Create trip lines (converts points into straight line)
64     conn.execute(
65         f"""
66         CREATE OR REPLACE TABLE trip_lines_2263 AS
67         SELECT
68             id,
69             trip_duration,
70             ST_MakeLine(
71                 ST_Transform(
72                     ST_Point(pickup_longitude, pickup_latitude),
73                     'EPSG:4326',
74                     'EPSG:2263',
75                     always_xy := true
76                 ),
77                 ST_Transform(
78                     ST_Point(dropoff_longitude, dropoff_latitude),
79                     'EPSG:4326',
80                     'EPSG:2263',
81                     always_xy := true

```

```

82         )
83     ) AS geom
84 FROM taxi_trips
85 WHERE
86     pickup_longitude IS NOT NULL
87     AND pickup_latitude IS NOT NULL
88     AND dropoff_longitude IS NOT NULL
89     AND dropoff_latitude IS NOT NULL
90     AND trip_duration > 0
91     """"
92 )
93
94 # Creates table with trip's speed by computing length of line (in miles) divided by time of trip and converted to hours.
95 # Distance (miles), time (hours) = miles per hour
96 conn.execute(
97     """"
98     CREATE OR REPLACE TABLE trip_speeds_2263 AS
99     SELECT
100         id,
101         geom,
102         (ST_Length(geom) / ?) / (trip_duration / ?) AS speed_mph
103 FROM trip_lines_2263
104     """"
105     [FEET_PER_MILE, SECONDS_PER_HOUR],
106 )
107
108 # Table using a spatial join (intersection between geometry of trip and geom of neighborhood)
109 # A trip belongs to a neighborhood if its line intersects neighborhood's polygon
110 conn.execute(
111     """"
112     CREATE OR REPLACE TABLE part2_neighborhood_speeds AS
113     SELECT
114         n.neighborhood,
115         AVG(t.speed_mph) AS average_mph
116 FROM trip_speeds_2263 AS t
117 INNER JOIN neighborhoods_2263 AS n
118     ON ST_Intersects(t.geom, n.geom)
119 WHERE t.speed_mph > 0 AND t.speed_mph <= ?
120 GROUP BY n.neighborhood
121 ORDER BY average_mph ASC
122     """"
123     [MAX_SPEED_MPH],
124 )
125
126 part2 = conn.execute("SELECT * FROM part2_neighborhood_speeds").fetchdf()
127
128 part2_path = BASE_DIR / "part2.csv"
129 part2.to_csv(part2_path, index=False)
130 print(part2.head(10))
131 print(f"Wrote {part2_path}")
132
133 # Build dataset for map visualization
134 # Convert SQL geometry to JSON format (usable for web map)
135 # Transform coordinate system back because Folium expects lat/lon format
136 map_rows = conn.execute(
137     """"
138     SELECT
139         n.neighborhood,
140         p.average_mph,
141         ST_AsGeoJSON(
142             ST_Transform(n.geom, 'EPSG:2263', 'EPSG:4326', always_xy := true)
143         ) AS geometry
144 FROM neighborhoods_2263 AS n
145 LEFT JOIN part2_neighborhood_speeds AS p USING (neighborhood)
146     """"
147 ).fetchall()
148
149 # Create GeoJson
150 features = [
151     {
152         "type": "Feature",
153         "properties": {
154             "neighborhood": neighborhood,
155             "average_mph": average_mph,
156         },
157         "geometry": json.loads(geometry),
158     }
159     for neighborhood, average_mph, geometry in map_rows
160 ]
161 geojson = {"type": "FeatureCollection", "features": features}
162
163 # Create map
164 speed_map = folium.Map(location=[40.7128, -74.0060], zoom_start=11, tiles="CartoDB positron")

```

```
165     folium.Choropleth(  
166         geo_data=geojson,  
167         name="Average speed",  
168         data=part2,  
169         columns=["neighborhood", "average_mph"],  
170         key_on="feature.properties.neighborhood",  
171         fill_color="RdYlGn",  
172         fill_opacity=0.75,  
173         line_opacity=0.25,  
174         legend_name="Average taxi speed (mph)",  
175         nan_fill_color="lightgray",  
176     ).add_to(speed_map)  
177     folium.GeoJson(  
178         geojson,  
179         name="Neighborhoods",  
180         tooltip=folium.GeoJsonTooltip(  
181             fields=["neighborhood", "average_mph"],  
182             aliases=["Neighborhood", "Average mph"],  
183             localize=True,  
184         ),  
185         style_function=lambda feature: {  
186             "fillOpacity": 0,  
187             "color": "#333333",  
188             "weight": 0.4,  
189         },  
190     ).add_to(speed_map)  
191     folium.LayerControl().add_to(speed_map)  
192  
193     map_path = BASE_DIR / "nyc_neighborhoods_speed_choropleth.html"  
194     speed_map.save(map_path)  
195     print(f"Wrote {map_path}")  
196  
197     conn.close()  
198  
199  
200 if __name__ == "__main__":  
201     main()
```